

Proof Buddy — Lemma Documentation

1. Use Instructions

1. Create a new proof using the name of the Lemma

Proof Buddy Courses All Proofs Help ▾

testStu Logout

Equational Reasoning

Name: consLen # Tag: demo

LHS Goal: `length (cons x B)` RHS Goal: `(+ 1 (length B))`

Current LHS: Current RHS:

START EQUATIONAL REASONING PROOF

2. Complete each line of the proof until LHS is equal to RHS

Proof Buddy Courses All Proofs Help ▾

testStu Logout

Equational Reasoning

Name: consLen # Tag: demo

CURRENT = LHS

Switch to Right Hand Side >>>

LHS Goal: `length (cons x B)` RHS Goal: `(+ 1 (length B))`

Current LHS: `(+ 1 (length B))` Current RHS: `(+ 1 (length B))`

002: `(if # 0 (+ 1 (length (rest (cons x B)))))`

003: `(+ 1 (length (rest (cons x B))))`

004: `(+ 1 (length B))`

005:

Num: **FILL VALUES**

3. Navigate to the **Check Proof Complete** button in the **Proof Utilities** dropdown in the top right corner
 - Once Proof Buddy verifies that the proof is complete, it can now be called upon in other proofs as a lemma/proven theorem.

The image displays two screenshots of the Proof Buddy web application interface.

Top Screenshot: The interface is titled "Equational Reasoning". It shows a proof in progress. The LHS Goal is `length (cons x B)` and the RHS Goal is `(+ 1 (length B))`. The Current LHS is `(+ 1 (length (rest (cons x B))))` and the Current RHS is `(+ 1 (length B))`. The proof steps are listed on the left: 002, 003, 004, and 005. The right sidebar contains a "Proof Utilities" menu with options: Definitions, View Rule Set, Check Current Proof (highlighted), New Proof, Discard Proof, and Download Proof. At the bottom, there is a "Num" field and a "FILL VALUES" button.

Bottom Screenshot: This screenshot shows the same interface after the proof is completed. A large, colorful "Proof Complete!!!!!!" overlay is centered on the screen, accompanied by a shower of confetti. The "Check Current Proof" button in the sidebar is still highlighted.

4. To apply a lemma, use the **"apply"** keyword followed by the name of the lemma
 - If you forget the name of a Lemma while in a proof, clicking "All Proof" to see list of all lemma, then go back to where you left off in the proof

Proof Buddy
Courses
All Proofs
Help

testStu
Logout

Equational Reasoning

Name
lemma demo

Tag
demo

Proof Utilities

CURRENT = LHS
Switch to Right Hand Side >>>

LHS Goal
(length (cons x (cons y null)))

Current LHS
(length (cons x (cons y null)))

RHS Goal
2

Current RHS
2

000

LHS Premise
Premise

(length (cons x (cons y null)))

001

LHS Rule

Proof Buddy
Courses
All Proofs
Help

testStu
Logout

All Equational Proofs

UPLOAD PROOF FROM A SAVED FILE

Equational Proofs

Search Equational proofs...
Search

Proof: lemma demo - demo

Goal: (length (cons x (cons y null))) = 2

Completed: False

Open Proof

Run Proof

Proof: consLen - demo

Goal: (length (cons x B)) = (+ 1 (length B))

Completed: True

Open Proof

Run Proof

Proof: premise test 3 (Assignment Copy) - test

Goal: (F 2 1) = 3

Completed: False

Open Proof

Run Proof

- For example: if the lemma is named "consLen" the rule entered should be **apply consLen**
 - On High Support:** parameter mapping is optional, but if given it must be correct (i.e. **apply consLen** $x \mapsto x$, $B \mapsto (\text{cons } y \text{ null})$)
 - On Low Support:** parameter mapping must be typed in manually and must be correct (i.e. **apply consLen** $x \mapsto x$, $B \mapsto (\text{cons } y \text{ null})$)
- Ensure that the highlighted expression matches what the premise of the lemma
 - Without parameter mapping inputted

Proof Buddy
Courses
All Proofs
Help

testStu
Logout

Equational Reasoning

Name
lemma demo

Tag
demo

Proof Utilities

CURRENT = LHS
Switch to Right Hand Side >>>

LHS Goal
(length (cons x (cons y null)))

Current LHS
(length (cons x (cons y null)))

RHS Goal
2

Current RHS
2

000

LHS Premise
Premise

(length (cons x (cons y null)))

001

LHS Rule

Num
001

LHS Rule
apply consLen

CLEAR LINE
GENERATE & CHECK
SUBSTITUTION

CANCEL

- With parameter mapping inputted

Proof Buddy Courses All Proofs Help ▾ testStu Logout

Equational Reasoning Name lemma demo # Tag demo Proof Utilities ▾

CURRENT = LHS Switch to Right Hand Side >>>

LHS Goal: `(length (cons x (cons y null)))` = RHS Goal: `2`

Current LHS: `(length (cons x (cons y null)))` Current RHS: `2`

000 LHS Premise Premise
`(length (cons x (cons y null)))`

001 LHS Rule
apply consLen $x \mapsto x, B \mapsto (cons\ y\ null)$

Num 001

CLEAR LINE GENERATE & CHECK SUBSTITUTION CANCEL

5. To finish the application of the lemma, click **Generate & Check** button, which will validate the application

- If valid, the next line in the proof will be available. Otherwise, there will be a toast message stating the error

Proof Buddy Courses All Proofs Help ▾ testStu Logout

Equational Reasoning Name lemma demo # Tag demo Proof Utilities ▾

CURRENT = LHS Switch to Right Hand Side >>>

LHS Goal: `(length (cons x (cons y null)))` = RHS Goal: `2`

Current LHS: `(+ 1 (length (cons y null)))` Current RHS: `2`

000 LHS Premise Premise
`(length (cons x (cons y null)))`

001 LHS Rule
apply consLen $B \mapsto (cons\ y\ null), x \mapsto x$

002 LHS Rule

Num FILL VALUES

***It should be noted that any completed proofs can be applied as a lemma**

2. Data Model

Unlike other rule applications or evaluations in Proof Buddy, lemmas do not have a Rule Type defined in `ERRuleset.py` and there is no separate **Lemma** model. There is no dedicated table for lemmas.

Lemmas are not built-in rules (e.g. **first-cons**) as they are user-created, user-specific, and stored in the database, rather than being hard-coded.

Lemmas are sourced from existing proof models:

- For a proof to be eligible to be used a lemma, it must have:
 - `is_complete=True`
 - `is_active=True`

- have a non-null `name` field
- The `EquationalProof` model is found in `/django_server/equational_reasoning_api/models.py`
- The `InductionProof` model is found in `/django_server/induction_api/models.py`

3. Lemma Lookup

Uses `_lookup_lemma()`, which is defined in both `equational_reasoning_api/views.py` and `induction_api/views.py`.

When it is given a lemma name and user, it queries the database for the **most recent completed proof** with that name and returns `(premise_str, conclusion_str, error)`.

4. Core Rule Class

The core rule class is called `LemmaRule` found in `ERRuleset.py`

`LemmaRule` extends the abstract `Rule` class and holds:

- `premise_tree / conclusion_tree` — parsed ASTs for LHS and RHS
- `param_names` — free variables detected in the premise

`isApplicable(targetNode, rawParams)` validates:

1. Correct number of `name=value` param assignments
2. Param names match the lemma's free vars
3. Each value parses to a valid AST node
4. Substituting params into the premise matches the target node structurally

`insertSubstitution(targetNode)` deep-copies the conclusion and substitutes all param bindings, returning the rewritten node.

5. Pure Engine Utilities

Three stateless functions (no Django ORM) in `expression_tree/LemmaApplicator.py`:

Function Name	Purpose
<code>extract_free_vars()</code>	finds alphabetic leaves not in the ruleset (free parameters)
<code>build_lemma_rule()</code>	parses premise/conclusion and returns a <code>LemmaRule</code> object
<code>validate_lemma_application()</code>	full validation and application pipeline

6. Dynamic Injection During Proof Steps

6.1 Process

When a student applies rule string `"apply <lemmaName> x=a, y=b"`, both the equational and induction proof engines:

1. Detect the `"apply"` prefix
2. Call `_lookup_lemma()` to fetch premise/conclusion from the DB

3. Call `build_lemma_rule()` to construct a `LemmaRule`
4. Temporarily inject it into `ruleSet['apply']`
5. Run normal rule application
6. Remove it from the ruleset immediately after

This temporary injection keeps rulesets clean and ensures lemmas don't persist across unrelated steps.

6.2 Parameter Inference

`_infer_params_for_rule()` found in `ERProofEngine.py` allows for a "high-support" mode.

In the "high-support" mode, the function walks thr premise tree and the highlighted target node in parallel, unifying free variables automatically.

As a result, students do no have to input `x=...`, `y=...` when applying a lemma.

7. API

There are no seperate lemma-only routes, it's integrated into existing rule endpoints:

Endpoint	Purpose
<code>POST /api/equational/apply-rule</code>	Apply any rule, including lemmas, to an equational proof
<code>POST /api/induction/apply-rule</code>	Apply any rule, including lemmas, to an induction proof
<code>GET /api/equation/list</code>	List proofs, which can be browsed as available lemmas

8. Frontend

Currently has minimal dedicated UI.

`CreateProof.js` has "Lemmas Allowed?" checkbox, which is a placeholder for future enforcement.

Students invoke lemmas by typing the rule string directly. There is currently no dedicated lemma-picker dropdown.

9. Lifecycle Summary

1. Student completes and names a proof

- Stored as `EquationalProof` / `InductionProof`
- Marked with `is_complete=True`

2. Student applies a lemma

```
apply myLemma x=a
```

3. Lemma is resolved and injected

```
_lookup_lemma()  
  ↓  
build_lemma_rule()  
  ↓  
inject into ruleSet
```

4. Lemma rule executes

```
LemmaRule.isApplicable()  
  ↓  
insertSubstitution()
```

5. Result returned

```
result node is returned  
  ↓  
lemma removed from the ruleSet
```